SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

CrickSense: Ball-by-Ball Analysis of a Cricket Broadcast from a Single Camera Feed

ADA23DLU02 - COMPUTER VISION LABORATORY

PROJECT REPORT

Submitted by

KABILASH.S-E0323036

PAVITRA.P-E0323038

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Data Analytics)

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai -600116

NOVEMBER, 2026

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

CrickSense: Ball-by-Ball Analysis of a Cricket Broadcast from a Single Camera Feed

ADA23DLU02 - COMPUTER VISION LABORATORY

PROJECT REPORT

Submitted by

KABILASH.S-E0323036

PAVITRA.P-E0323038

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Data Analytics)

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai -600116

NOVEMBER, 2026

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this project report **"CrickSense: Ball-by-Ball Analysis of a Cricket Broadcast from a Single Camera Feed"** is the Bonafide record of work done by **"KABILASH.S-E0323036"** and **"PAVITRA.P-E0323038"** who carried out the work under my supervision.

Signature of the Supervisor

Dr. Nirmala B
Assistant Professor,
Department of Artificial Intelligence and Data Analytics, Sri Ramachandra Faculty of Engineering and Technology,
SRIHER, Porur, Chennai-600 116.

Signature of External Examiner

Evaluation Date: 3 November 2026

ABSTRACT

Measuring a cricket match from its television broadcast is hard: the ball is a few pixels across and moves several metres between frames, the only useful camera stands far behind the bowler and pans and zooms through every delivery, and the picture cuts away for most of each ball. **CrickSense** analyses a broadcast from one camera feed, with no sensors and no access to the broadcaster's data. It calibrates the pitch automatically from its painted creases, reads the score bar over whole matches, detects and tracks the players and names them by role, finds each delivery's bounce, length, line and speed, and keeps the match state by the Laws of Cricket.

Calibration placed 7 of 8 crease corners within 3.3 px of hand-clicked points and calibrated 14 of 20 random clips with none wrong; the score bar was read over two whole matches with all four final scores exact. On three hand-verified clips, two never seen during development, all 21 deliveries and outcomes were right, and the score engine agreed with the score bar on 230 of 230 checks. Speed is the broadcaster's radar when shown, else a time-of-flight estimate with a median error of 5.7% against radar on the final - while the pixel-displacement method of the earlier prototype was measured at 1-72 km/h against radar readings of 81-146 km/h. Commentary is written by a language model from measured facts only, every line fact-checked, and spoken by a local text-to-speech model over the crowd, with the broadcast's own commentators removed by source separation. A live mode on one RTX 5060 Laptop GPU reports each ball's outcome about 0.5 s after the score bar settles and the radar speed in about 0.1 s, and names the players on screen.

The system is evaluated against an answer key verified by hand against the broadcast, re-run after every change, and the report documents what was measured and dropped: speed from pixel motion, runs from the batters running, wides by ball position, front-foot no-balls and shot names from vision-language models. The main limits are the accuracy of the speed estimate for pace bowling at 25 frames per second and live player naming over a whole match, where the camera references go stale as the light changes.

Keywords: cricket, broadcast video, camera calibration, homography, object detection, multi-object tracking, ball tracking, OCR, live video analysis, fact-checked commentary.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**i**
	LIST OF FIGURES	**iii**
	LIST OF TABLES	**iv**
1	**INTRODUCTION**	**1**
	1.1 Background of the Study	1
	1.2 Problem Statement	1
	1.3 Objectives of the Study	1
	1.4 Scope and Limitations	2
	1.5 Significance of the Study	2
	1.6 Organization of the Report	2
2	**LITERATURE REVIEW**	**3**
	2.1 Background	3
	2.1.1 Review of Related Work	3
	2.2 Research Gaps	4
3	**PROPOSED METHODOLOGY**	**5**
	3.1 Introduction	5
	3.2 Proposed System/Methodology	5
	3.2.1 Overall Workflow Diagrams	5
	3.3 Module Description	7
	3.3.1 Algorithms / Models Used	8
	3.3.2 Evaluation Metrics	9
	3.4 Summary	9
4	**SYSTEM IMPLEMENTATION**	**10**
	4.1 Introduction	10
	4.2 Hardware and Software Specifications	10
	4.2.1 Tools / Libraries Used	10
	4.3 Implementation Details	10
	4.3.1 Dataset Collection and Descriptions	19
	4.4 Pseudocode / Code Snippets	20
	4.5 Summary	21
5	**CONCLUSION AND FUTURE WORK**	**22**
	5.1 Summary of the Research	22
	5.2 Conclusion Drawn from the Results	22
	5.3 Contributions of the Study	22
	5.4 Future Work	22
	REFERENCES	**23**

LIST OF FIGURES

Figure 3.1: CrickSense workflow.....6

Figure 4.1: Automatic calibration.....11

Figure 4.2: The score bar's fields (blue) and the striker's bat icon (orange).....11

Figure 4.3: Score-bar ball counts seen over the whole final as each fix went in (121 balls in each innings).....12

Figure 4.4: Offline overlay, ball 12.1 (Nortje to Kohli).....12

Figure 4.5: Time-of-flight speed estimate against the broadcaster's radar for the 11 deliveries with both.....13

Figure 4.6: Where every delivery pitched in the five analysed overs of both matches (length and line from ball.json).....14

Figure 4.7: Pitch map of the 6-minute clip.....14

Figure 4.8: Ball 11.5, Axar's six.....16

Figure 4.9: Voice heard between our commentary lines.....16

Figure 4.10: Live mode on Bumrah's over, 1 s behind the source.....17

Figure 4.11: Live latency per event on Bumrah's over.....17

Figure 4.12: Live mode on the whole final, calibrated live from the creases.....18

Figure 4.13: Time per pass for the 6-minute clip (RTX 5060 Laptop GPU).....18

LIST OF TABLES

Table 2.1: Comparison of related work....4
Table 2.2: Research gaps and how CrickSense addresses them....4
Table 3.1: The passes of the offline pipeline....6
Table 3.2: Models in CrickSense....8
Table 3.3: Evaluation metrics....9
Table 4.1: Hardware and software....10
Table 4.2: Calibration results....11
Table 4.3: Score bar over whole matches....11
Table 4.4: Ball results on the three answer-key clips....13
Table 4.5: Score engine checks against the score bar....15
Table 4.6: Sample commentary (Super 8, Starc to Rohit)....15
Table 4.7: Live mode results....17
Table 4.8: Approaches measured and dropped....19
Table 4.9: Video data and evaluation clips....20

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Cricket is followed mostly through its television broadcast. For every ball the broadcaster shows a picture from a camera behind the bowler, a score bar and often the speed of the delivery from a radar. Everything else analysts discuss - where the ball pitched, its line, who faced it, what the batter did - is either measured by multi-camera ball-tracking installations that only top matches have (Hawk-Eye [2]) or noted by hand. Computer vision in sports has been surveyed by Thomas et al. [1]; for broadcast cricket, Abbas et al. [3] segment deliveries and track the ball within a delivery shot, and TrackNet [4] and TTNet [5] track small fast balls from a fixed camera.

Broadcast cricket is hard for measurable reasons. The ball is 6-15 pixels across in a 1080p frame and moves 3-6 m between two frames at 25 frames per second. The main camera stands far behind the bowler with a long lens, so along the pitch one pixel covers about 3.5 cm at the bowler's crease. The camera pans and zooms hard in every delivery, and the director cuts to replays and close-ups, so the main view is on screen for only a few seconds per ball. Registering a frame to the field is usually done by fitting a homography to field lines [6], [7], [8]; a cricket pitch is small in the frame, but its painted creases are fixed by the Laws [9], which makes a direct geometric fit possible.

1.2 Problem Statement

Given one broadcast video and nothing else - no sensors, no access to the broadcaster's data - produce for every ball who bowled and who faced, where it pitched (length and line), how fast it was, what the batter did and what the ball produced; keep a match state that follows the Laws of Cricket and agrees with the broadcaster's score; present it as an annotated video with spoken commentary; and run the time-critical part live. Every number shown must be measured or clearly marked as an estimate.

1.3 Objectives of the Study

1. **O1 - Calibration.** Calibrate the broadcast camera to the pitch automatically, every frame, without hand-clicked points.
2. **O2 - Score bar.** Read the broadcaster's score bar reliably over a whole match and use it as the source of outcomes.
3. **O3 - Players.** Detect and track the players and give them roles (striker, non-striker, bowler, keeper, umpire) and names.
4. **O4 - Ball.** Find each delivery's bounce, length, line and speed, and measure their accuracy against radar and a hand-checked answer key.

5. **O5 - Match state.** Keep score, strike and bowler figures by the Laws, checked against the bar after every ball.
6. **O6 - Commentary.** Generate commentary and speech from measured facts only, and render the annotated video.
7. **O7 - Live.** Run outcomes, speed and named players live, and measure the latency of each.

1.4 Scope and Limitations

The system works on one broadcast feed. It was built and tested on the ICC Men's T20 World Cup 2024 final (India v South Africa, 5 h 21 min, 1080p, 25 fps) and checked on a second match (Australia v India, Super 8). The score-bar reader needs a layout file per broadcaster's graphics; calibration, detection and the rest are automatic. What the broadcast does not show cannot be measured: runs taken by running (the picture cuts away within 1.8 s of the shot on 14 of 15 balls), the field setting, wides by position and front-foot no-balls (margins of 1-5 cm against 3.5 cm per pixel) are read from the score bar or left out, each decision backed by a measurement (Table 4.8).

1.5 Significance of the Study

The project shows how much of a cricket match can be measured from what every viewer already has - the broadcast - on a laptop GPU, and documents how reliable each measurement is, including what failed. A pipeline like this can annotate archive footage for coaches and analysts, produce pitch maps and ball-by-ball data for matches that have no tracking installation, and give viewers live statistics. Its evaluation discipline - held-out clips, a hand-verified answer key, negative results reported - makes it useful for teaching computer vision on real broadcast data.

The work also shows where classical computer vision still matters next to deep learning. The learned detectors find people and the ball, but the measurements come from geometry - a homography fitted to painted lines, a camera followed by optical flow, a time of flight - and from rules that check them. The data needed is small: 94 hand-reviewed frames fine-tuned the ball model, the score-bar font was harvested from the broadcast itself, and nothing else was labelled. The system runs on a consumer laptop with an 8 GB GPU, uses free or local models for language and speech, and keeps no personal data beyond the public broadcast.

1.6 Organization of the Report

Chapter 2 reviews related work and the research gaps. Chapter 3 describes the proposed methodology: the system, its workflow and each module, the models used and the evaluation metrics. Chapter 4 covers the implementation: hardware and software, each module as built with its measured results, the dataset and pseudocode of the key algorithms. Chapter 5 concludes and outlines future work.

CHAPTER 2: LITERATURE REVIEW

2.1 Background

Sports video analysis. Computer vision in sports has been surveyed by Thomas et al. [1], who separate broadcast-based analysis - one or a few uncalibrated cameras whose operators pan, zoom and cut - from installed systems with dedicated calibrated cameras. Broadcast analysis is cheaper and covers every televised match, but must cope with camera motion, cuts and graphics; installed systems measure more precisely but exist only at the top level of the game.

Small fast objects. A cricket ball is a few pixels across and blurred by motion. Single-frame detectors of the YOLO family [10], [11] find it in a still frame; methods that look at several consecutive frames, such as TrackNet [4] and TTNet [5], exploit its motion to find it when a single frame is ambiguous. All of them return pixel positions; turning those into metres on the pitch requires the camera's calibration.

Camera calibration and field registration. A homography maps the plane of the field to the image [6]. In sports it is usually fitted to detected field lines, either by searching over camera parameters [7] or by learning from synthetic views of the field [8]. Following a moving camera between calibrated frames is done by matching image features - ORB [14] with RANSAC [15] - or by sparse optical flow [16], [17]. A cricket pitch has few lines, but they are fixed by the Laws [9].

People, text, language and audio. Multi-object trackers such as ByteTrack [12] and BoT-SORT [13] associate detections of people over time; keypoint models estimate body pose [11]. Score graphics can be read with OCR such as Tesseract [18]. Language models write fluent commentary but may state facts that are not in their input [19], and source-separation models such as Hybrid Transformer Demucs [20] separate voices from the rest of a soundtrack.

2.1.1 Review of Related Work

The works closest to this project fall into four groups (Table 2.1). **Installed ball tracking:** Hawk-Eye [2] reconstructs the ball in 3D from several calibrated high-frame-rate cameras around the ground; it is accurate but needs installation at the venue and is not available from the broadcast. **Ball tracking in broadcast or fixed-camera video:** Abbas et al. [3] segment cricket deliveries with a YOLO-based detector and track the ball within each delivery shot; TrackNet [4] predicts a heatmap of a tiny fast ball from consecutive frames of a fixed camera, and TTNet [5] adds event spotting for table tennis. None of them calibrates the pitch, so none measures length, line or speed in metres. **Field registration:** Homayounfar et al. [7] and Chen and Little [8] register broadcast frames of football to a field model, which works because the field and its many lines fill the picture; a cricket pitch is a small strip in the frame. **Players, actions and language:** ByteTrack [12] and BoT-SORT [13] track people, but with camera motion estimated from image features rather than from a calibration; Sen et al. [24] classify

batting shots with a CNN and a GRU trained on labelled clips; and language models that write commentary are prone to stating facts that are not in their input [19].

Table 2.1: Comparison of related work

Reference	Year	Method	Data	Limitation for this project
Owens et al. [2]	2003	calibrated multi-camera 3D ball reconstruction	installed high-speed cameras	needs the venue's installation
Abbas et al. [3]	2022	YOLO delivery segmentation, ball tracking	cricket delivery clips	no calibration: no length, line or speed
Huang et al. [4]	2019	heatmap CNN over consecutive frames	badminton, tennis (fixed camera)	fixed camera; labelled sequences
Voeikov et al. [5]	2020	ball detection and event spotting	table tennis (fixed camera)	fixed camera; another sport
Homayounfar et al. [7]	2017	field localisation, structured model	football broadcast	large field with many lines
Chen and Little [8]	2019	calibration trained on synthetic views	football broadcast	needs a field model and training
Zhang et al. [12]	2022	tracking by associating every detection	pedestrian benchmarks	camera motion from image features
Aharon et al. [13]	2022	BoT-SORT: tracking with camera-motion compensation	pedestrian benchmarks	no pitch geometry or roles
Sen et al. [24]	2021	CNN + GRU batting-shot classification	labelled cricket shot clips	needs a labelled shot set
Ji et al. [19]	2023	survey of hallucination in language generation	NLG benchmarks	commentary must be fact-checked

2.2 Research Gaps

Table 2.2 links each gap to its evidence and to how this project addresses it; the objectives in Section 1.3 answer them one by one.

Table 2.2: Research gaps and how CrickSense addresses them

Gap	Evidence	How this project addresses it
G1. Ball analysis needs an installation, or stops at the ball within a delivery shot	[2], [3]	the full chain from broadcast pixels to a checked match state
G2. Field registration targets large fields with many lines	[7], [8]	calibration from the regulation creases, precision first
G3. Speed from a single broadcast camera is rarely checked against radar	[3], [4]	time-of-flight estimate validated against the broadcaster's radar
G4. Trackers estimate camera motion from image features and know no roles	[12], [13]	camera motion from the calibration; roles from pitch geometry
G5. Language models state facts not in their input	[19]	every commentary line fact-checked against measured facts
G6. Evaluations seldom use held-out answer keys or report failures	[1]	hand-verified answer key, unseen clips, approaches dropped with their measurements

CHAPTER 3: PROPOSED METHODOLOGY

3.1 Introduction

CrickSense turns a cricket broadcast into ball-by-ball data and an annotated video. It follows three design principles. **Measure from the broadcast, or say it is not measured:** outcomes the broadcast does not show come from the score bar and are labelled so; our own speed estimate is drawn as "≈ ... estimate". **Precision first:** a wrong calibration silently corrupts everything after it, so doubtful frames are rejected; a missing result is visible, a wrong one is not. **Independent passes joined by files with strict schemas:** each pass runs, fails and is timed alone; its JSON output is the next pass's input.

3.2 Proposed System/Methodology

The system has an offline pipeline of ten passes, a live mode that runs the time-critical parts on a stream as it plays, and a verification layer that checks every result before it is shown. One command, python main.py <video>, analyses a clip in full, or a whole match: it checks the team sheet, reads the score bar once, prints both final scores, lists every over and analyses the chosen ones. For a clip, the data flows as follows:

1. **Geometry.** Calibration finds the camera views from the painted creases and gives every frame its image-to-pitch homography (camera.json).
2. **Reading.** The score-bar reader turns the bar into a time series of trusted states: score, overs, batters, striker, bowler and radar speed (scoreboard.json).
3. **Perception.** With each frame's homography, the players pass places every person on the pitch and names them, the ball pass finds each delivery and its bounce, and the pose pass reads the striker (players.json, ball.json, pose.json).
4. **Reasoning.** Fusion joins all of it into events and keeps the match state by the Laws, checked against the bar after every ball (events.json, match.json).
5. **Language and output.** Fact sheets become checked commentary and speech; the renderer draws the overlay video, pitch map and wagon wheel; the store and API publish the results.

The live mode runs the same ideas on a stream with three workers that never wait for each other, and the verification layer checks each result - physics, the score bar, the answer key - before it is shown.

3.2.1 Overall Workflow Diagrams

Figure 3.1 shows the workflow as built, in three bands; Table 3.1 lists what each offline pass reads and writes.

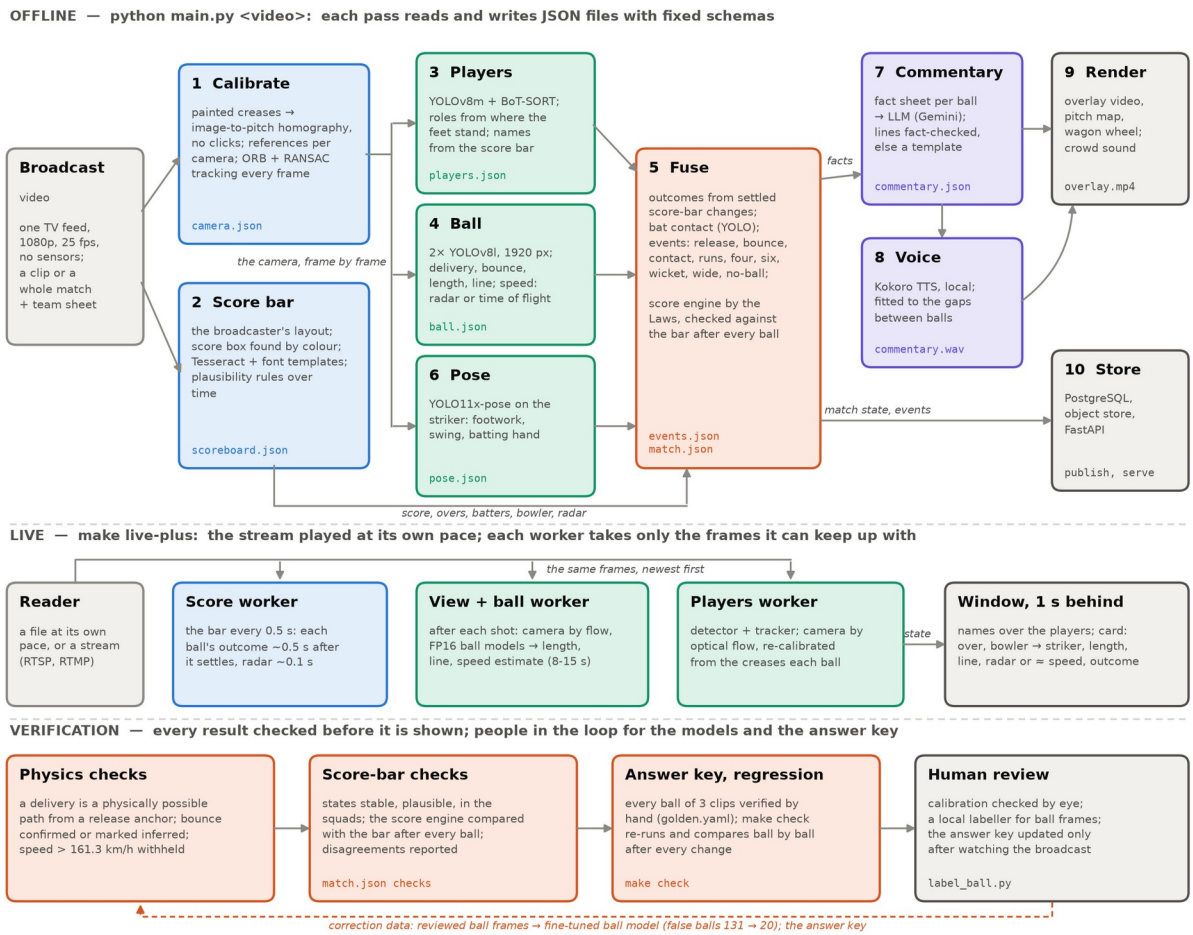


Figure 3.1: CrickSense workflow: the offline passes and the files they exchange (top), the live path (middle) and verification (bottom)

Table 3.1: The passes of the offline pipeline

Pass	Input	Method	Output
1 Calibrate	video	crease-based homography, references, ORB tracking	camera.json
2 Score bar	video	layout fields, score box by colour, font templates	scoreboard.json
3 Players	video, camera	YOLOv8m, BoT-SORT, roles by pitch position	players.json
4 Ball	video, camera	two YOLOv8l models, delivery finder, bounce	ball.json
5 Fuse	all above	events, outcomes, score engine by the Laws	events.json, match.json
6 Pose	video, players	YOLO11x-pose on the striker	pose.json
7 Commentary	facts	LLM with fact check, template fallback	commentary.json
8 Voice	commentary	Kokoro text-to-speech	commentary.wav
9 Render	all	overlay, pitch map, wagon wheel, Demucs	overlay.mp4
10 Store / API	all	PostgreSQL, object storage, FastAPI	database, HTTP API

3.3 Module Description

Calibration. Every pitch carries the same painted creases: popping creases 1.22 m in front of the stumps, return creases 1.32 m either side of middle, stumps 20.12 m apart [9]. White-line candidates (top-hat filter, pitch-coloured ground on both sides) give hypotheses for the four popping-crease corners, each defining a homography H from image pixels to pitch metres [6]. A hypothesis is scored by how much of the regulation crease pattern lands on paint, plus whether the stumps stand where the middle-stump base projects; homographies no real camera could produce are rejected. A camera view is accepted only with two strong agreeing frames (score ≥ 0.78) or three, because a batter on a crease can fool one. Every other frame is registered to the references with ORB [14] and RANSAC [15] and smoothed with a Savitzky-Golay filter [21].

Score bar. A layout file gives each field's rectangle (Figure 4.2); the pink score box is also found by colour, because the bar slides and the chase shows a compact variant. Names are read with Tesseract [18] and matched to the squads; score digits are read glyph by glyph against templates of the broadcaster's font. A state is trusted only if it is stable (seen twice), plausible (never backwards; at most one ball per 15 s and 8 runs per ball) and recognised.

Players. People are detected with a COCO-trained YOLOv8m [10], [11], [23] and tracked with BoT-SORT [13], whose camera-motion compensation uses the per-frame calibration. Roles come from where each player's feet stand on the pitch plus movement (the bowler runs in, the umpire stands still), accumulated over time and assigned jointly with the Hungarian algorithm; names come from the score bar and the team sheet.

Ball. Two detectors run at 1920 px: one finds each delivery, one fine-tuned on 94 hand-reviewed frames of this broadcast gives the track. A delivery is a physically constrained path from a release anchor; the bounce is where the ball's ground point turns, giving length (from the striker's stumps) and line (from the middle-stump line). Speed is the broadcaster's radar when shown, else the time of flight from release to bounce; estimates above 161.3 km/h are withheld.

Events and score engine. Outcomes come from settled score-bar changes (legal ball, wide, no-ball, boundary, wicket); a ball-and-over state machine keeps score, strike, extras and bowler figures by the Laws and is compared with the bar after every ball.

Pose. YOLO11x-pose [11] on the striker gives footwork (only when the feet are visible), the swing (wrist speed relative to the hips, so camera motion cancels) and the batting hand.

Commentary, voice and render. For each ball a fact sheet is built; a language model writes a line from it and every line is checked - each number must be in the facts, only the players involved named, no boundary or wicket the facts lack, a word budget to fit the gap - else a template is used, a guard against hallucination [19]. Lines are spoken by the local Kokoro

model [22] over the stadium sound, from which the broadcast's own commentators are removed with Demucs [20]. The renderer draws named players, the ball path held on the pitch while the camera moves, a delivery card, a pitch map and a wagon wheel.

Live mode. A reader plays the stream at its own pace and offers each frame to independent workers; a busy worker gets the newest frame, never a backlog. A score worker reads the bar every 0.5 s; a view-and-ball worker processes each main-view shot when it ends; a players worker names the players on the newest main-view frame, following the camera by optical flow [16], [17] and re-calibrating each delivery from its creases. The picture is shown 1 s behind the source so boxes sit on the frame they were found on.

Verification. Physics checks (a possible path, a confirmed or inferred bounce, no impossible speed), score-bar checks after every ball, a hand-verified answer key re-run after every change, and human review: calibrations checked by eye, ball frames reviewed in a local labeller that fine-tuned the ball model.

3.3.1 Algorithms / Models Used

Table 3.2: Models in CrickSense

Model	Task	Input	Origin
YOLOv8l ball_best	find deliveries	1920 px	trained by the team (cricket ball)
YOLOv8l ball_ft	track the ball	1920 px	fine-tuned on 94 frames of this broadcast
YOLOv8 bat_best	bat contact	crops	trained by the team
YOLOv8m (COCO)	people	1280 px	pre-trained, used as is
YOLO11x-pose	striker keypoints	crops	pre-trained, used as is
Tesseract + font templates	score bar	fields	templates harvested automatically
Gemini 3.5 Flash-Lite	commentary lines	fact sheets	API, free tier
Kokoro-82M int8	speech	text	local, no key
htdemucs	remove TV commentators	audio	pre-trained (MIT licence)

The key algorithms are classical geometry around the learned detectors: homography fitting from the creases, ORB matching with RANSAC, pyramidal Lucas-Kanade optical flow for the live camera, Gaussian role evidence with Hungarian assignment, and a time-of-flight speed estimate v = d / (t_bounce - t_release), with d the distance from the release point (just past the bowler's popping crease, about 2.1 m up) to the bounce. Pseudocode is given in Section 4.4.

3.3.2 Evaluation Metrics

Table 3.3: Evaluation metrics

Module	Metric	Reference
Calibration	crease-corner error (px); clips calibrated, wrongly calibrated	hand-clicked corners; projected creases checked by eye
Score bar	ball counts seen of 121; final score exact	official scorecard
Ball	deliveries and bounces found; length (m)	hand-checked answer key
Speed	absolute percentage error of the estimate	broadcaster's radar
Outcomes, score engine	outcomes right; checks agreeing with the bar	answer key; score bar
Players	stable identities per player appearance; role and name correct	hand check
Commentary	lines passing the fact check; voice heard between our lines	fact sheet; Demucs on the result
Live	latency after the bar settles; real-time factor	wall clock

Every result is measured on clips that were not used to build or tune the component measured, against an answer key verified by hand against the broadcast (every ball: over, score, striker, bowler, radar); where the reference is the broadcaster's own data, that is stated.

3.4 Summary

The method replaces what the broadcast cannot show with what it does show - the score bar and the radar - and measures the rest geometrically from the painted creases. Learned detectors find the people and the ball; classical geometry turns them into pitch coordinates, lengths and speeds; rules of physics and the Laws check them; and a fact-checked language model only describes what was measured.

CHAPTER 4: SYSTEM IMPLEMENTATION

4.1 Introduction

CrickSense is a Python package with one command per pass, a single entry point and a live mode. Every threshold lives in YAML configuration files with the measurement behind it recorded in a decisions log. Quality is guarded by 200 automated tests and a regression check (make check) that re-runs three hand-verified clips and compares them ball by ball. This chapter gives the environment, each module as built with its measured results, the data, and pseudocode of the key algorithms.

4.2 Hardware and Software Specifications

Table 4.1: Hardware and software

Item	Specification
Laptop	AMD Ryzen 9 8940HX, 32 GB RAM
GPU	NVIDIA GeForce RTX 5060 Laptop GPU, 8 GB
Operating system	Linux (Omarchy, Arch-based)
Language	Python 3.11 (conda environment)
Deep learning	PyTorch 2.14 (CUDA 13), Ultralytics 8.4

4.2.1 Tools / Libraries Used

- **Ultralytics YOLOv8 / YOLO11** [11]: detection of the ball, bat and people; pose keypoints.
- **OpenCV 5.0, NumPy, SciPy**: ORB, RANSAC homographies, optical flow, image processing, Hungarian assignment.
- **Tesseract** [18]: score-bar text; the score digits use templates harvested from the broadcast.
- **FFmpeg**: decoding, clip cutting, encoding the overlay; ffplay for the live window.
- **Gemini API** (any OpenAI-compatible API): commentary lines; **Kokoro-82M** [22]: local speech; **Demucs 4.1** [20]: source separation.
- **PostgreSQL, SQLAlchemy, FastAPI**: storage and the HTTP API; **pytest**: 200 automated tests.

4.3 Implementation Details

Each module was implemented as a pass and measured on held-out clips; the results are given with each module.

Calibration. Figure 4.1 shows a calibrated frame and Table 4.2 the results. Calibration is precise where it succeeds and fails visibly where it cannot - on 20 random clips none was

calibrated wrongly. Its weakness is time: a reference matches the broadcast for minutes, not hours, as the light moves, which the live mode handles by re-calibrating from the creases.

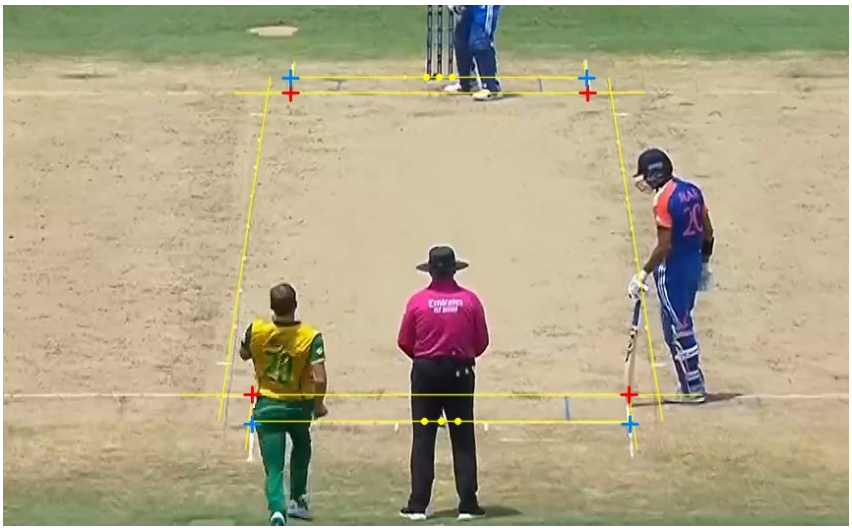


Figure 4.1: Automatic calibration: the regulation creases projected through the fitted homography lie on the painted creases

Table 4.2: Calibration results

Measure	Result
Automatic vs hand-clicked corners	7 of 8 corners within 3.3 px
20 random clips of the final (no clicks)	14 calibrated, none wrongly
Bumrah's over: references per camera 1 → 6	frames tracked 206 → 884; deliveries 5/6 → 6/6
Half-resolution matching (rejected)	1.9× faster; corner error 1.8 → 3.1 px median
A reference 51 min away (live)	1 of 291 main-camera frames matched
A reference 2 min away	222-405 matched points (80 needed)

Score bar. Figure 4.2 shows the fields read and Table 4.3 the whole-match results. Finding the score box by colour raised South Africa's ball counts seen from 90 to 115, and the font templates to 117 (Tesseract read "11-1" as "1-1", "11" or "-1" under every setting; the templates score 250/250 leave-one-out).



Figure 4.2: The score bar's fields (blue) and the striker's bat icon (orange); read here: Axar 38 (26), Kohli* 41 (35), 93-3 after 12 overs, Nortje 0-4

Table 4.3: Score bar over whole matches

Match	Innings	Ball counts seen	Final score read
T20 WC final	India	118 of 121	176-7 (exact)
T20 WC final	South Africa	117 of 121	169-8 (exact)
Super 8	India	121 of 121	205-5 (exact)
Super 8	Australia	119 of 121	181-7 (exact)

Figure 4.3 shows how each change raised the ball counts seen over the whole final: the plausibility rules (a misread that jumped ahead had blocked every genuine state after it), then the score box found by colour for the chase's moving bar, then the font templates.

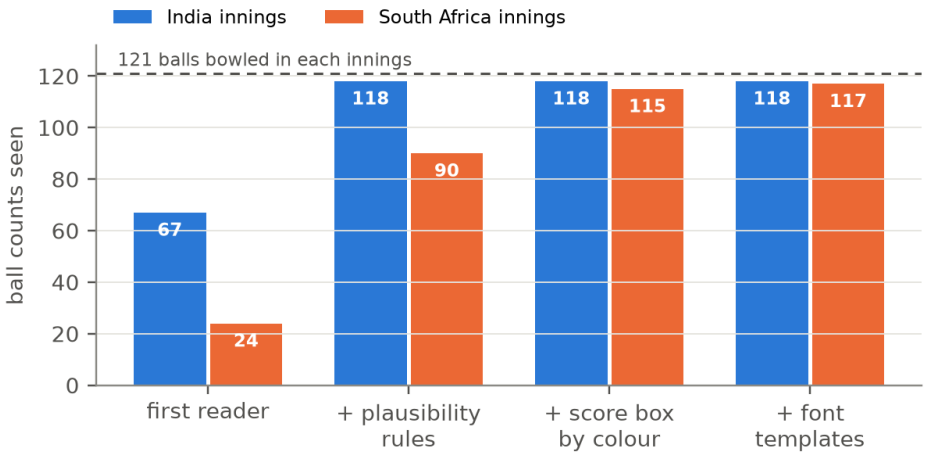


Figure 4.3: Score-bar ball counts seen over the whole final as each fix went in (121 balls in each innings)

Ball, bounce and speed. Figure 4.4 shows the offline overlay and Table 4.4 the ball results. The two ball models are complementary: either alone found 10 or 9 deliveries and 8 or 6 bounces on the development clip, together 10 and 9; false balls on crease marks fell from 131 to 20 after fine-tuning; a bounce checked by hand measured 4.77 m against 4.8 m.

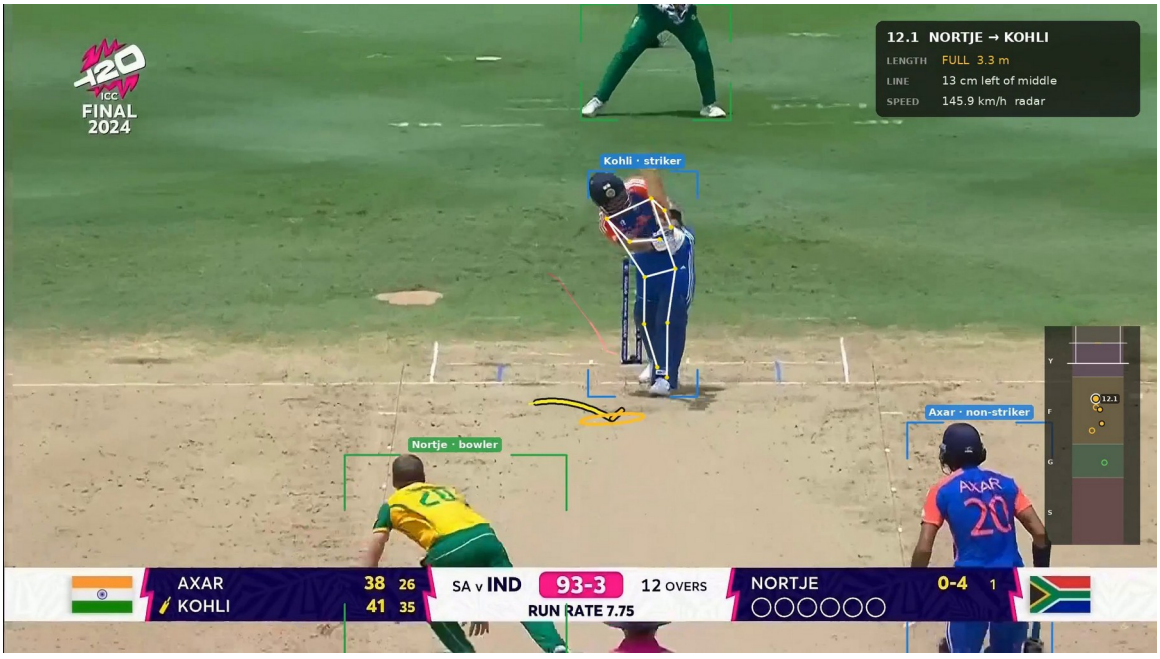


Figure 4.4: Offline overlay, ball 12.1 (Nortje to Kohli): named players, the striker's pose, the ball's path and bounce, and the delivery card with the radar speed

Table 4.4: Ball results on the three answer-key clips

Clip	Deliveries	Bounces	Radar shown	Outcomes
India 11.1-12.4 (development)	10/10	9	6 of 10	10/10
South Africa 7.1-7.5 (unseen)	5/5	3	5 of 5	5/5
Bumrah's over 2 (unseen)	6/6	6	6 of 6	6/6
Total	21/21	18	17 of 21	21/21

Speed is the clearest case of measurement changing the design. The earlier prototype computed speed from the ball's pixel displacement through the ground homography; measured, it gave 1-72 km/h against radar readings of 81-146 km/h, because an airborne ball is not on the ground plane. The time-of-flight estimate (Figure 4.5) has a median error of 5.7% on the final and 8.3% over both matches (11 deliveries). Its error grows with pace: a fast ball reaches the bounce about 8 frames after release, so one frame is about 13% of the speed, and a release seen late makes it wrong - Starc's delivery came out 39% low.

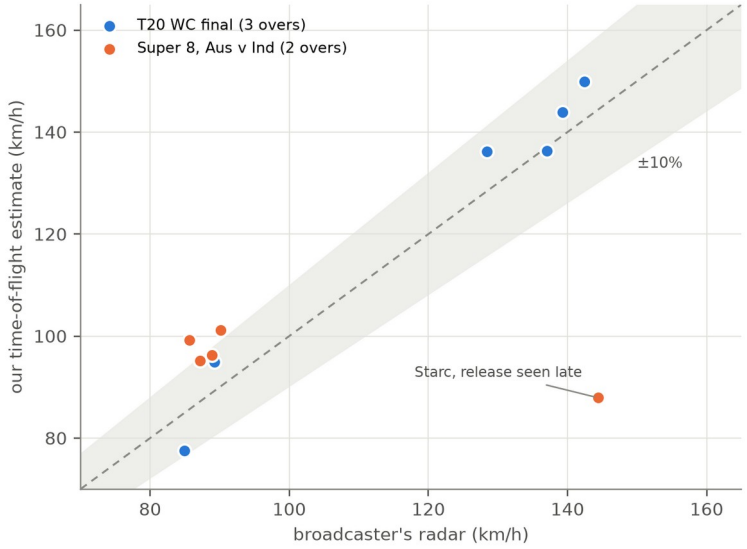


Figure 4.5: Time-of-flight speed estimate against the broadcaster's radar for the 11 deliveries with both; the band is ±10%



Figure 4.6: Where every delivery pitched in the five analysed overs of both matches (length and line from ball.json)

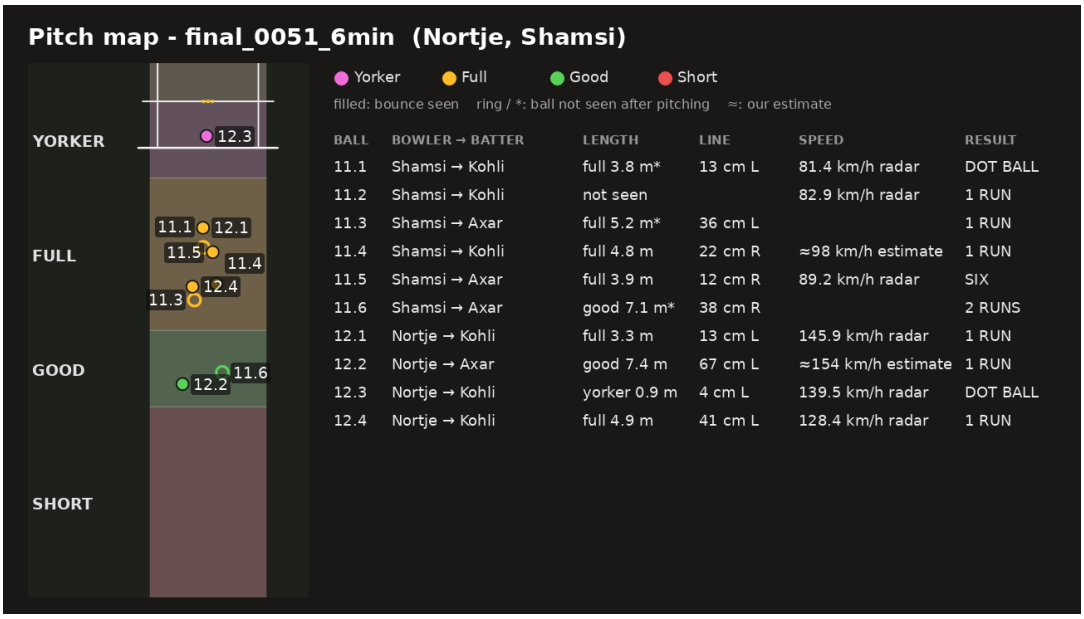


Figure 4.7: Pitch map of the 6-minute clip: each ball's bounce, length zone, line, speed and its source, and outcome

Figure 4.6 places every bounce found in the five analysed overs: the pace overs cluster on a good length outside off stump, while Starc's yorker and the spinners' full balls sit near the stumps.

Outcomes and score engine. All 21 outcomes of the answer-key clips are right, including a six, two fours, a wicket and an extra, and the score engine agreed with the bar on every check

(Table 4.5). Three faults found on the unseen clips were fixed and re-measured on all clips: one reference per camera missed the wicket ball; Tesseract froze the score at 7-1; a second "delivery" 2 s after a real one stole its radar reading.

Table 4.5: Score engine checks against the score bar

Clip	Balls	Checks agreeing	Notes
India 11.1-12.4	10	88 / 88	a six, strike rotation
South Africa 7.1-7.5	5	40 / 40	spin, unseen
Bumrah's over 2	6	54 / 54	a wicket and a four, unseen
Super 8, Starc's over	7 + a wide	48 / 48	6, 6, 4, 6, 0, wide, 6 = 29

Players and pose. 51 stable identities for 51 player appearances over 12 delivery shots (the earlier prototype's tracker produced 534 identities for about 13 people). The swing was "played" on all 8 balls with runs off the bat and "left" on the one ball not hit; the batting hand was right on 10 of 10 balls.

Commentary and voice. Table 4.6 shows lines written by the model from the facts; every one passed the fact check. Figure 4.8 shows a line in the rendered video and Figure 4.9 the removal of the broadcast's own commentators. Shot type was tried with vision-language models and switched off after measurement: asked for a shot name, models called almost every shot a "sweep"; asked for posture, the free models called three different swings "cover drive". Removing the broadcast's commentators with two passes of Demucs left no detectable voice between our lines (213 s → 0 s of 290 s) with the crowd only 0.4 dB quieter.

Table 4.6: Sample commentary (Super 8, Starc to Rohit)

Ball	Facts	Line written by the model
2.1	full, 138.3 km/h radar, SIX, 12-1	Starc bowls a full delivery at 138.3 kph, Rohit plays it from the crease for SIX!
2.2	full, 140.2 km/h radar, front foot, SIX	Starc clocks 140.2 kph, full delivery, Rohit on the front foot plays it for SIX!
2.3	full, 138.9 km/h radar, FOUR	Starc at 138.9 kph, it looks full, Rohit plays it and it goes for FOUR!

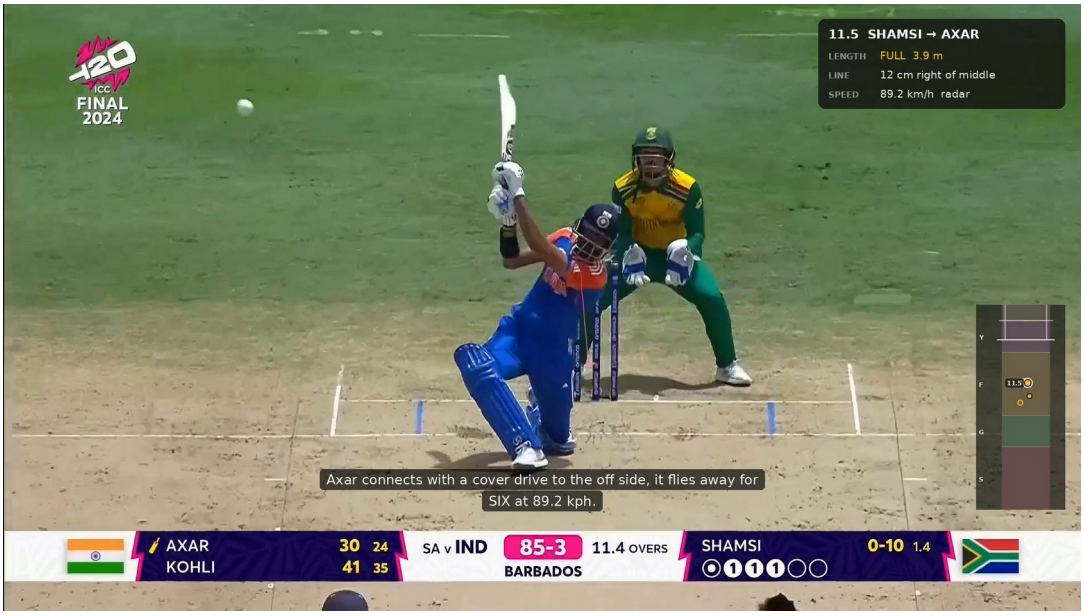


Figure 4.8: Ball 11.5, Axar's six: the delivery card with the radar speed and the spoken line shown as a caption

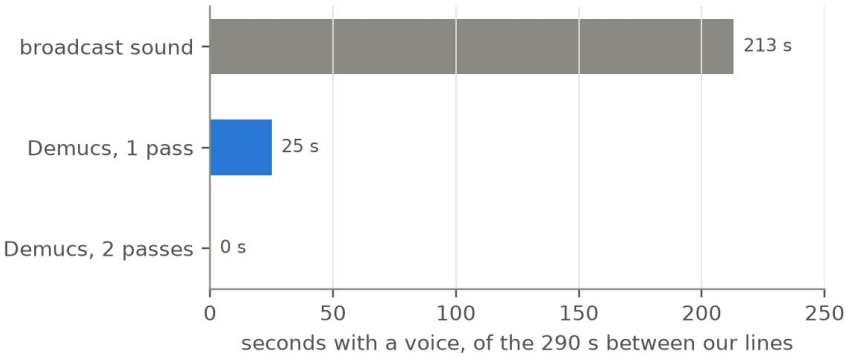


Figure 4.9: Voice heard between our commentary lines: the broadcast sound, and the crowd track after one and two passes of Demucs

Live mode. Table 4.7 gives the live results, Figure 4.11 the latency of each kind of result, and Figures 4.10 and 4.12 live frames on one over and on the whole final. Half precision made the ball model 3.3 times faster (51 ms per frame at 1920 px instead of 168, the same detections on 48 of 48 frames); following the camera by optical flow placed players within 0.25 m (median) of the offline calibration. On a whole match each delivery must be re-calibrated from its creases in the first second of its shot, which a shared laptop GPU does not always manage, so players are named on about half the deliveries.

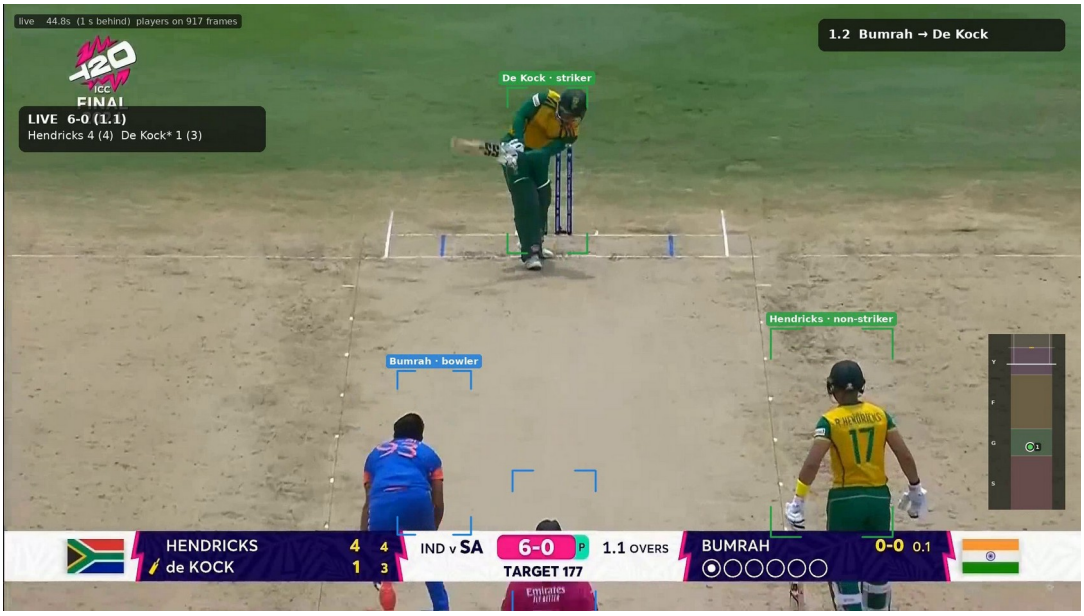


Figure 4.10: Live mode on Bumrah's over, 1 s behind the source: striker, non-striker, bowler and umpire named; the card for ball 1.2 with its radar speed

Table 4.7: Live mode results

Measure	Bumrah's over	Whole final, first 7 min
Kept pace with the source	327 s for 320 s	425 s for 420 s
Each ball's outcome	6/6, 0.5 s after the bar settled	every ball
Radar speed	about 0.1 s	about 0.1 s
Length, line, estimate	every delivery, 12-23 s after it	most deliveries
Players named	every delivery	about half the deliveries

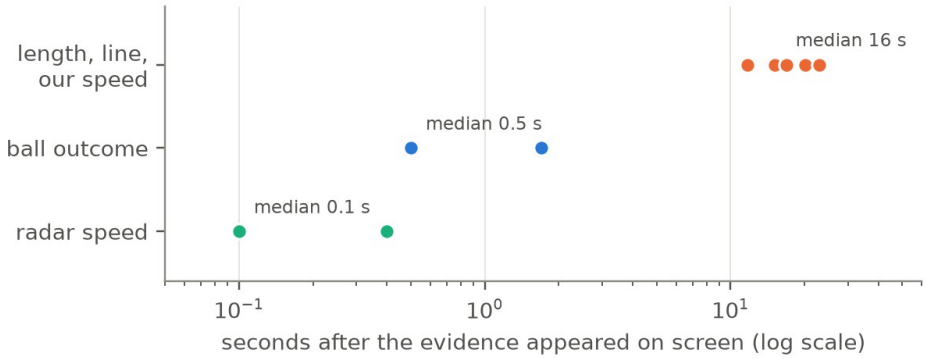


Figure 4.11: Live latency per event on Bumrah's over: how long after the evidence appeared on screen each result was shown

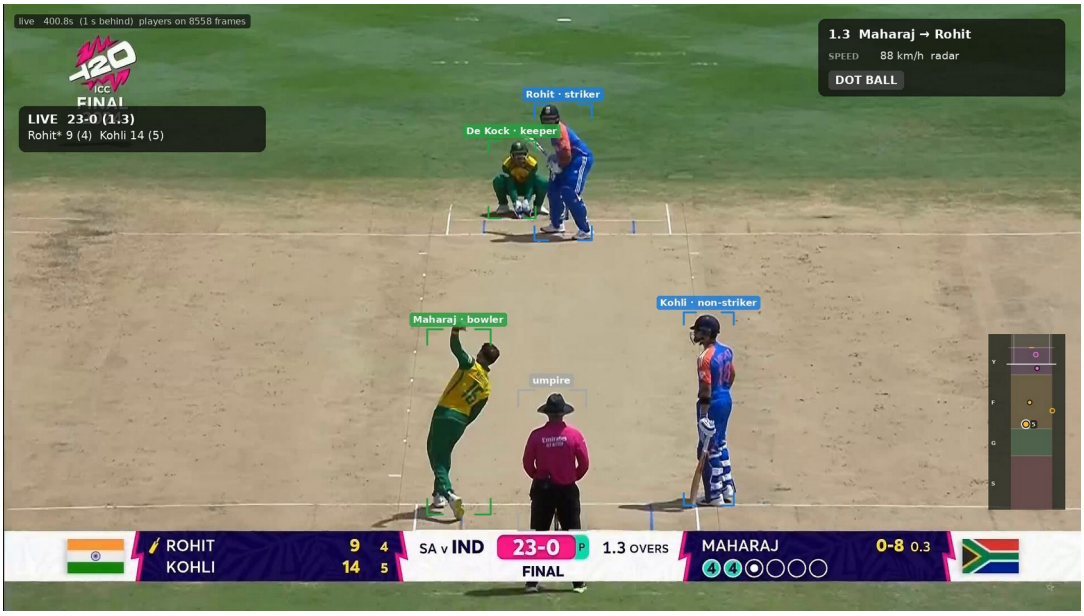


Figure 4.12: Live mode on the whole final, calibrated live from the creases: Rohit (striker), Kohli (non-striker), Maharaj (bowler), de Kock (keeper) and the umpire

Processing time. Figure 4.13 gives the time of each offline pass on the 6-minute clip. Every pass but calibration runs faster than real time; calibration matches every frame - including the 93% that are replays and close-ups - at full resolution, which the live mode's flow-followed camera avoids.

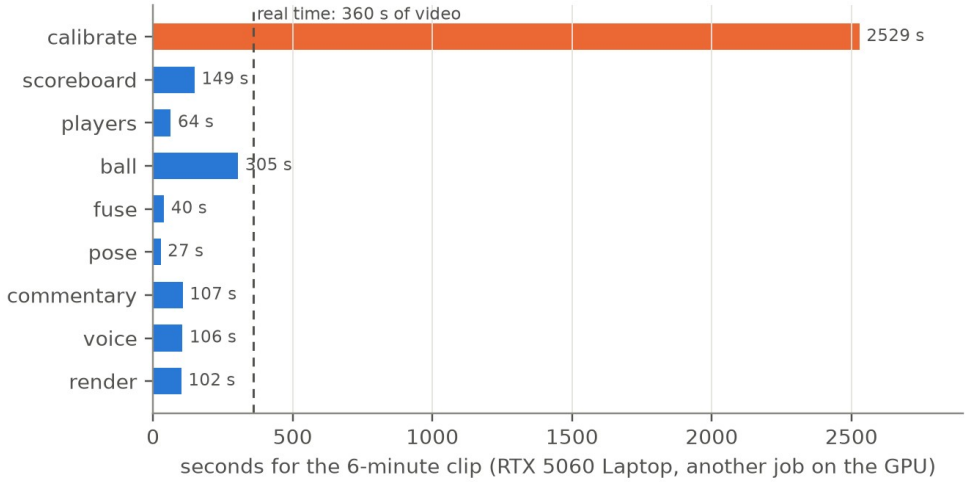


Figure 4.13: Time per pass for the 6-minute clip (RTX 5060 Laptop GPU)

Structured outputs (JSON). Every pass writes JSON with a fixed schema; these records from Bumrah's over show the chain for the 1.1 delivery and the 1.3 wicket. ball.json holds each delivery's geometry, events.json the time-stamped events, and match.json the state after every ball with each check against the score bar:

```
ball.json  - delivery 0 (over 1.1)
{"release_frame": 298, "release_t": 11.90, "detections": 12,
 "bounce_t": 12.20, "bounce_confirmed": true,
 "length_m": 7.40, "line_m": -0.08, "length_class": "good",
 "radar_speed_kph": 139.3, "speed_estimate_kph": 143.9,
 "track_model": "ball_ft.pt"}

events.json - the wicket (over 1.3)
{"frame_id": 2412, "t": 96.50, "type": "WICKET", "source": "scoreboard",
 "payload": {"bowler": "BUMRAH", "striker": "HENDRICKS",
             "over": "1.3", "score": "7-1"}, "confidence": 0.9}

match.json - the state after it
{"over": "1.3", "runs": 7, "wickets": 1, "legal_balls": 9,
 "striker": "MARKRAM", "non_striker": "DE KOCK", "bowler": "BUMRAH",
 "batters": {"HENDRICKS": [4, 5], "DE KOCK": [1, 4], "MARKRAM": [0, 0]},
 "bowlers": {"BUMRAH": [0, 3, 1]},
 "checks": {"score": true, "wickets": true, "overs": true,
            "striker as predicted": true, "Markram runs/balls": true,
            "De Kock runs/balls": true, "Bumrah figures": true},
 "notes": ["wicket: incoming batter and strike from the score bar"]}
```

What did not work. Table 4.8 lists approaches that were measured and dropped.

Table 4.8: Approaches measured and dropped

Approach	Measurement	Decision
Speed from pixel displacement	1-72 km/h against radar 81-146	radar; time of flight
Runs from the batters running	pitch on screen ≤ 1.8 s after the shot on 14 of 15 balls	score bar
Wides by ball position	legal balls measured 1.3-2.75 m wide	score bar
Front-foot no-ball	3.5 cm per pixel; margins 1-5 cm	score bar
Retrained 5-class player model	0.895 mAP50 on its split; almost no detections here	COCO model + geometry
Shot names from a VLM	"sweep" for almost every shot; unstable between runs	switched off

4.3.1 Dataset Collection and Descriptions

Two full match broadcasts were used (MP4, 1920 × 1080, 25 fps): the ICC T20 World Cup 2024 final (5 h 21 min) and the Super 8 match Australia v India (4 h 57 min). Clips were cut with frame-accurate offsets. The three answer-key clips were verified ball by ball by hand; two of them were never seen during development. 94 frames were hand-reviewed to fine-tune the ball model; nothing else was labelled.

Table 4.9: Video data and evaluation clips

Clip	Content	Length	Use
Final, India 11.1-12.4	Shamsi (spin), Nortje (pace) to Kohli, Axar	6 min	development, answer key
Final, South Africa 7.1-7.5	Kuldeep (spin) to Stubbs, de Kock	3 min	unseen, answer key
Final, South Africa over 2	Bumrah (pace); a wicket and a four	5 min	unseen, answer key
20 random clips of the final	around balls found by the score bar	10-60 s	benchmark
Super 8, overs 3 and 9	Starc to Rohit; Kuldeep, a wicket	7 min each	second match

4.4 Pseudocode / Code Snippets

The live pipeline (simplified):

```
procedure LIVE_PLUS(stream, calibration)
  bank <- reference views of the calibration      // shared by all workers
  start workers SCORE, VIEW_BALL, PLAYERS
  for each frame f, read at the source's own pace:
      offer f to every worker              // a busy worker keeps the newest
      draw the frame from 1 s ago with the latest live state

SCORE(f):     read the score bar of f
              if a state settled: emit OUTCOME; if a new radar: emit SPEED
PLAYERS(f):   H <- FLOW_CAMERA(f)                  // image -> pitch
              if H: boxes <- detector + tracker; feet <- H(box bottoms)
                    roles <- Hungarian(evidence(feet, motion))
                    names <- score bar and team sheet
VIEW_BALL(s): for the frames of shot s in order:
                  H <- FLOW_CAMERA(frame); ball candidates (FP16)
                  stop 12 frames after a confirmed bounce
              d <- FIND_DELIVERY; emit BOUNCE(length, line, speed)
```

FLOW_CAMERA follows the camera cheaply and re-calibrates from the creases when no reference matches; a crease calibration is trusted only when a second one agrees:

```
def homography(frame):
    flow_H = H_prev @ inv(flow(prev, frame)) if H_prev else None  # ~10 ms
    if flow_H is not None and since_try < 6:
        return flow_H
    if not shows_pitch_strip(frame):           # outfield, crowd shots
        return flow_H
    if references_match(frame):                # 40-330 ms
        return reference_H
    if flow_H is not None and since_full < 60:
        return flow_H
    c = calibrate_from_creases(frame)          # 0.3-0.5 s
    if c.score >= 0.78 and agrees(c.H, candidate, 20):   # px
        bank.add(frame, c.H)                   # confirmed: a new reference
        return c.H
    candidate = c.H                            # carried by the flow
```

4.5 Summary

The implementation meets its objectives on the measured clips: automatic calibration (7 of 8 corners within 3.3 px; none wrong on 20 clips), whole-match score reading (four final scores exact), all 21 deliveries and outcomes of the answer-key clips, 230 of 230 score-engine checks, speed by radar or a time-of-flight estimate (5.7% median on the final), fact-checked commentary with the broadcast's commentators removed, and a live mode that reports outcomes about 0.5 s after the bar and names the players. Its limits are measured too: pace speed estimates, stale references over a whole match, and partial live naming over a whole match.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Summary of the Research

CrickSense analyses a cricket broadcast from one camera feed: it calibrates the pitch from its painted creases, reads the score bar over a whole match, names the players by role, finds each delivery's bounce, length, line and speed, keeps the match state by the Laws, writes and speaks fact-checked commentary, renders an annotated video, and runs the time-critical parts live on one laptop GPU. Every component was measured on clips not used to build it, and the measurements decided what was kept.

5.2 Conclusion Drawn from the Results

- All 21 deliveries and outcomes of the answer-key clips were found and right, and the score engine agreed with the bar on 230 of 230 checks over five overs of two matches; both matches' final scores were read exactly.
- Automatic calibration from the creases was precise and never wrong on 20 random clips, but references go stale within minutes as the light moves.
- Ball speed from a single behind-the-bowler camera is meaningful only as time of flight (5.7% median error on the final, worse for pace); the broadcaster's radar is the better source when shown.
- Outcomes and radar run live with about half a second of latency; length and line arrive 8-23 s after the ball; players are named live over an over, partly over a whole match.

5.3 Contributions of the Study

- Click-free calibration of broadcast cricket from the regulation creases, with a precision-first acceptance rule.
- A score-bar reader with font templates and plausibility rules that reads whole matches exactly.
- A measured evaluation of ball speed from broadcast video, including why pixel-displacement speed fails.
- Fact-checked commentary with speech, and a live architecture on one laptop GPU (drop-frame workers, a flow-followed camera, crease re-calibration).

5.4 Future Work

- 50 fps footage and labelled frames of the ball's first flight, for pace speed and missed deliveries.
- Faster crease calibration, so that players are named live through a whole match.
- A labelled answer key for shot types; score-bar layouts for other broadcasters.

REFERENCES

[1] G. Thomas, R. Gade, T. B. Moeslund, P. Carr and A. Hilton, "Computer vision for sports: Current applications and research topics," Computer Vision and Image Understanding, vol. 159, pp. 3-18, 2017.

[2] N. Owens, C. Harris and C. Stennett, "Hawk-eye tennis system," in Proc. Int. Conf. Visual Information Engineering (VIE), 2003, pp. 182-185.

[3] K. Abbas, M. Saeed, M. I. Khan, K. Ahmed and H. Wang, "Deep-learning-based computer vision approach for the segmentation of ball deliveries and tracking in cricket," arXiv:2211.12009, 2022.

[4] Y.-C. Huang, I.-N. Liao, C.-H. Chen, T.-U. İk and W.-C. Peng, "TrackNet: A deep learning network for tracking high-speed and tiny objects in sports applications," in Proc. IEEE AVSS, 2019.

[5] R. Voeikov, N. Falaleev and R. Baikulov, "TTNet: Real-time temporal and spatial video analysis of table tennis," in Proc. IEEE/CVF CVPR Workshops, 2020.

[6] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge University Press, 2004.

[7] N. Homayounfar, S. Fidler and R. Urtasun, "Sports field localization via deep structured models," in Proc. IEEE CVPR, 2017.

[8] J. Chen and J. J. Little, "Sports camera calibration via synthetic data," in Proc. IEEE/CVF CVPR Workshops, 2019.

[9] Marylebone Cricket Club, The Laws of Cricket, 2017 Code (3rd ed., 2022).

[10] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, 2016.

[11] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLO (YOLOv8, YOLO11)," software, 2023-2024. https://github.com/ultralytics/ultralytics

[12] Y. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," in Proc. ECCV, 2022.

[13] N. Aharon, R. Orfaig and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," arXiv:2206.14651, 2022.

[14] E. Rublee, V. Rabaud, K. Konolige and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in Proc. IEEE ICCV, 2011.

[15] M. A. Fischler and R. C. Bolles, "Random sample consensus," Communications of the ACM, vol. 24, no. 6, pp. 381-395, 1981.

[16] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in Proc. IJCAI, 1981.

[17] J. Shi and C. Tomasi, "Good features to track," in Proc. IEEE CVPR, 1994.

[18] R. Smith, "An overview of the Tesseract OCR engine," in Proc. ICDAR, 2007.

[19] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, 2023.

[20] S. Rouard, F. Massa and A. Défossez, "Hybrid transformers for music source separation," in Proc. IEEE ICASSP, 2023.

[21] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," Analytical Chemistry, vol. 36, no. 8, 1964.

[22] hexgrad, "Kokoro-82M: an open-weight text-to-speech model," model card, 2025. https://huggingface.co/hexgrad/Kokoro-82M

[23] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Proc. ECCV, 2014.

[24] A. Sen, K. Deb, P. K. Dhar and T. Koshiba, "CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit," Sensors, vol. 21, no. 8, p. 2846, 2021.